Ukesoppgaver - Uke 7

Nevrale nettverk - IN1160

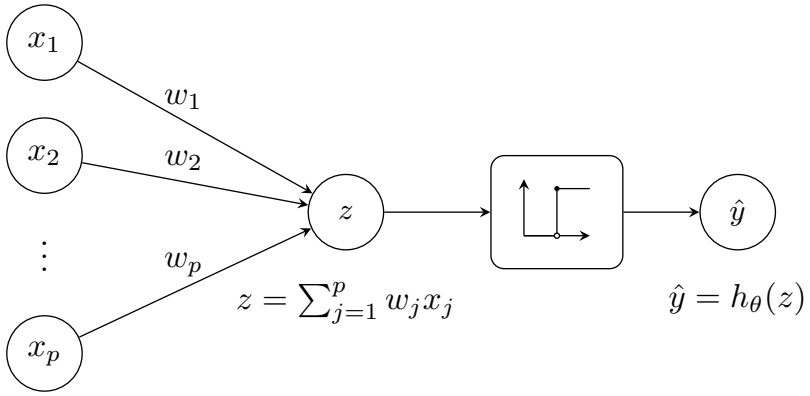
I ukens ukesoppgaver skal vi regne litt på perseptroner og nevrale nettverk, noe tilsvarende det vi har gjort i de to forrige ukene med lineær og logistisk regresjon. Perseptronet er, på samme vis som modeller basert på logistisk regresjon, en modell for å gjøre binær klassifisering. Perseptronet er i dag sjeldent brukt, siden logistisk regresjon stort sett fungerer bedre. Det er allikevel verdt å bruke litt tid til å se på perseptronet, ettersom det har vært historisk viktig for utviklingen av nevrale nettverk og at det kan gi pedagogisk forståelse av disse.

Oppgave 7.2 er laget for å gi en guidet gjennomgang av utregningen i foroverfasen i et nevralt nettverk, mens tapsfunksjonen "kryssentropi" for mer enn to klasser er dekket i oppgave 7.3. Ukens oppgaver er noe lengre enn andre uker, så vær forberedt på å bruke mer tid på dem.

I oblig 2b kommer dere til å implementere foroverfasen i nevrale nettverk, sammen med blant annet metrikker og aktiveringsfunksjoner, og bruke det til å klassifisere bilder av håndtegnede sifre. Bakoverfasen er også viktig å kjenne til, men selve utregningen for bakoverfasen er ikke pensum i dette faget. Oblig 2b går litt igjennom hvordan bakoverfasen fungerer.

Oppgave 7.1 - Perseptron

Vi kan starte med å fremstille perseptron som et diagram, tilsvarende det vi har gjort for lineær og logistisk regresjon.



Figur 1: Illustrasjon av en perseptron.

Som vi ser av diagrammet i Figur 1 fungerer perseptronet veldig likt som logistisk regresjon. Den største forskjellen i foroverfasen er at perseptronet bruker step-funksjonen h_θ, mens logistisk regresjon bruker sigmoid. Step-funksjonen er definert ved:

$$h_\theta(x) \;=\; \begin{cases} 1, & \text{if } x \geq \theta, \\ 0, & \text{ellers.} \end{cases}$$

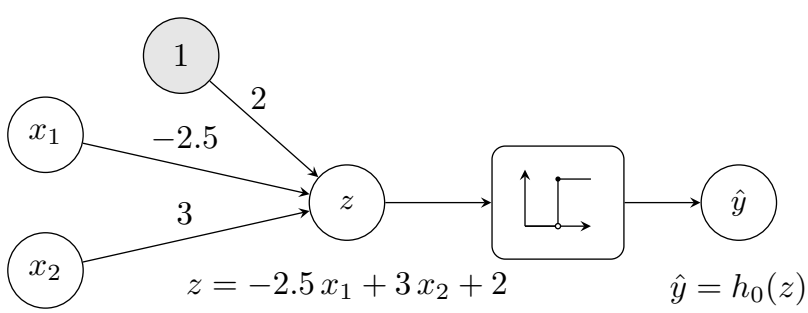
Her har vi en parameter θ (den greske bokstaven "theta") som bestemmer en terskel for når funksjonen skal slå ut. Er x-verdien som sendes inn over eller lik terskelverdien, får vi 1 ut fra funksjonen, ellers får vi ut 0. For vår del kommer θ ganske ofte til å være satt til 0, mens vi heller legger til et konstantledd (bias). Vi justerer altså heller summen til z enn å justere terskelen bestemt av θ

Perseptronet klassifiserer altså ved å først regne ut den vektede summen z (tilsvarende som med lineær og logistisk regresjon) og deretter sende dette gjennom step-funksjonen h_0.

Merk at for perseptron og nevrale nettverk bruker vi som regel terminologien *vekter* for parametere som ganges med inputen og *bias* eller *konstantledd* for parametere som plusses på. For lineær og logistisk regresjon bruker man som regel *koeffisienter* for begge deler. Dette har opphav i at lineær og logistisk regresjon har blitt utviklet i statistiske fagområder, mens perseptron og nevrale nettverk har i større grad opphav innenfor informatikk (selv om alle er statistiske modeller).

I motsetning til lineær og logistisk regresjon kan vi ikke bruke gradient descent på perseptron, siden funksjonen h_θ ikke er deriverbar. Perseptronet har allikevel en oppdateringsregel som gjør at den iterativt prøver å finne bedre vekter.

Oppgave 7.1.1 - Klassifisering med perseptron



Figur 2: Eksempel på en perseptron.

La oss nå se på et spesifikt eksempel for et perseptron og regne ut hvordan modellen klassifiserer. Vi bruker modellen som representeres med diagrammet i Figur 2.

Modellen vår tar altså input med to trekk, og har vektene $\mathbf{w} = (w_1, w_2) = (-2.5, 3)$ og konstantleddet 2.

Regn ut klassifiseringene $\hat{y}_i$ for modellen med følgende input:

a) $\mathbf{x}_1 = (1, 1)$

b) $\mathbf{x}_2 = (3, 0.5)$

c) $\mathbf{x}_3 = (0, -0.7)$

Oppgave 7.1.2 - Oppdateringsregelen for perseptron

Under trening oppdateres vektene i perseptronet seg etter hvert datapunkt, som vi skal se på i oppgave 7.1.3. Vektene vil altså ikke forbli det samme dersom modellen predikerer feil.

Perseptroner oppdaterer vektene sine med forskjellen $\Delta w_j = \eta(y - \hat{y})x_{i,j}$, der η er læringsraten og $x_{i,0} = 1$ for konstantleddet. Deretter oppdateres vektene med regelen $w_j^{\text{new}} \leftarrow w_j^{\text{old}} + \Delta w_j$. La oss nå bruke litt tid på å forstå hvorfor denne formelen kan bidra til bedre vekter. Læringsraten η vil alltid være positiv.

a) Anta at $\hat{y} = y$, altså at modellen klassifiserer riktig. Hva skjer med vektene i oppdateringen? Synes du dette gir mening?

b) Anta nå $\hat{y} = 1$ mens $y = 0$ og inputen $\mathbf{x}$ er positiv. Hvordan blir vektene justert nå? Gir dette intuitivt mening? Hva om inputen var negativ?

c) Til slutt la oss anta at $\hat{y} = 0$ og $y = 1$. Hva skjer med vektene, avhengig om inputen er positivt eller negativt? Hva med når inputen er lik 0? Gir dette mening?

Oppgave 7.1.3 - Oppdatering av parameterne

La oss bruke prediksjonene fra oppgave 7.1.1 til å regne ut de nye vektene og nøyaktighet (accuracy) av prediksjonene. Anta at de sanne verdiene tilhørende inputene fra oppgave 7.1.1 er $\mathbf{y} = (y_1, y_2, y_3) = (0, 0, 1)$. Merk at vi oppdaterer vektene etter

hvert datapunkt. Det vil si at om modellen predikerer feil for et datapunkt, vil vektene bli oppdatert før vi predikerer neste datapunkt.

Anta at læringsraten $\eta = 0.5$.

a) Gitt prediksjonene vi fikk i 7.1.1, hva er modellens nøyaktighet?

b) Hva blir vektene etter første prediksjon, tilsvarende inputen $\mathbf{x}_1 = (1, 1)$?

c) Med de oppdaterte vektene fra b), hva predikerer modellen for det andre datapunktet, $\mathbf{x}_2 = (3, 0.5)$? Hva blir vektene oppdatert til etter denne klassifiseringen?

d) Regn ut hva modellen predikerer med de nye vektene for det tredje datapunktet, $\mathbf{x}_3 = (0, -0.7)$, og hvordan vektene blir oppdatert.

e) Basert på de tre datapunktene og deres klassifisering, hva blir nøyaktigheten til modellen?

Oppgave 7.2 - Nevralt nettverk for klassifisering

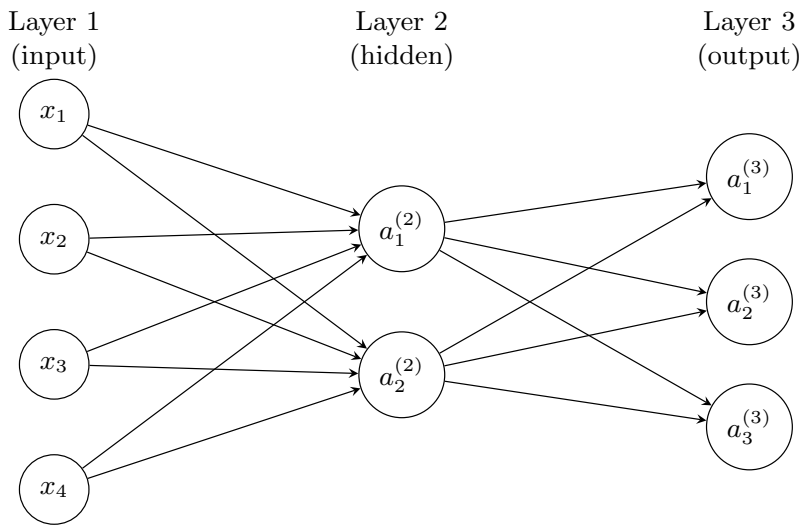
Vi skal nå begynne å jobbe med nevrale nettverk. Nevrale nettverk er ekstremt fleksible og nyttige modeller som blir brukt i mange moderne KI-modeller. I praksis brukes en rekke triks for hvordan nettverkene er bygget opp og trener, men vi skal begynne å se på den enkleste formen for nevrale nettverk.

Det er sterkt anbefalt å se de tre første videoene i denne introduksjonene til nevrale nettverk av 3Blue1Brown.

Vi begynner med et eksempel på et nevralt nettverk som har fire noder i inputlaget (som tilsvarer fire trekk (features) i inputen), ett skjult lag med to noder, og et output lag med tre noder (som tilsvarer tre klasser blant de sanne verdiene). Dette kan visualiseres som i Figur 3.

Merk at illustrasjonen for nevrale nettverk er litt forskjellig fra illustrasjonene av modellene vi har sett på tidligere. Modellen inneholder konstantledd (bias) og aktiveringsfunksjoner som er implisitt fra modellen. I tillegg skriver vi ikke navnene på alle parameterne, siden det fort blir rotete.

For hvert lag har vi nå en *matrise* (2-dimensjonalt array) av vekter. Hver node i et lag har en vekt til hver node i det neste laget. I tillegg har alle nodene unntatt det første laget et konstantledd (som er implisitt fra diagrammet). Disse lagene har også en *ikke-lineær aktiveringsfunksjon*. En ikke-lineær aktiveringsfunksjon er nødvendig for at modellen skal kunne lære ikke-lineære sammenhenger i dataene.



Figur 3: Illustrasjon av et nevralt nettverk.

La oss se litt nærmere på notasjonen. Vi har valgt å bruke notasjonen fra boken Neural Networks and Deep Learning, som er samme notasjon brukt i videoene til 3Blue1Brown. Det er dessverre mange ulike måter å bruke notasjon for nevrale nettverk på, så vær forberedt på at notasjonen kan være forskjellig i andre kontekster.

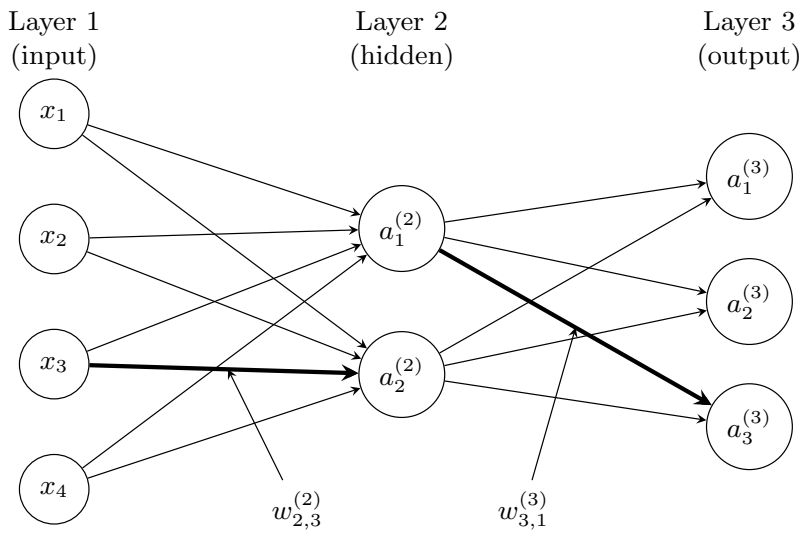
Nodene for hvert lag er kalt *aktiveringene* (activations), der vi i et lag nummer l med n_l antall noder noterer dette med $\mathbf{a}^{(l)}$. Merk at l her *ikke* er en eksponent, vi bruker isteden denne plassen øverst til høyre til å notere hvilket lag noe tilhører, siden vi bruker nederst til høyre til å notere hvilke noder det tilhører. Aktiveringene for det første laget er lik inputen vårt $\mathbf{x}$. Konstantleddene i et lag l noteres ved $\mathbf{b}^{(l)}$, der vi har et konstantledd for hver node i laget, bortsett fra første lag.

Vi bruker *kolonnevektorer* til å notere aktiveringene og konstantleddene. Dette er fordi det gjør at matrise- og vektormultiplikasjonen med vektene blir riktig. I eksemplet vårt fra diagrammet får vi da:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \qquad \mathbf{a}^{(2)} = \begin{bmatrix} a_1^{(2)} \\ a_2^{(2)} \end{bmatrix} \qquad \mathbf{a}^{(3)} = \begin{bmatrix} a_1^{(3)} \\ a_2^{(3)} \\ a_3^{(3)} \end{bmatrix} \qquad \mathbf{b}^{(2)} = \begin{bmatrix} b_1^{(2)} \\ b_2^{(2)} \end{bmatrix} \qquad \mathbf{b}^{(3)} = \begin{bmatrix} b_1^{(3)} \\ b_2^{(3)} \\ b_3^{(3)} \end{bmatrix}$$

Til slutt har vi vektene, som noteres i en matrise. I eksemplet vårt har vi vektene $W^{(2)}$ fra lag én til to, og vektene $W^{(3)}$ fra lag to til tre (det er vanlig å bruke store bokstaver for å representere matriser). Vekten $w_{j,k}^{(l)}$, som er vekten i rad nummer j og kolonne nummer k i $W^{(l)}$, vil gå fra node k i lag nummer $l-1$ til node j i lag l.

Figur 4 viser en illustrasjon av dette.



Figur 4: Indeksering av vekter i et nevralt nettverk.

Det kan virke litt omvendt at vekt i rad j og kolonne k går fra node k til j istedenfor node j til k, men dette gjør at notasjonen for matrise- og vektormultiplikasjon blir enklere. Vi kan notere vektene i eksemplet vårt i sin helhet med følgende:

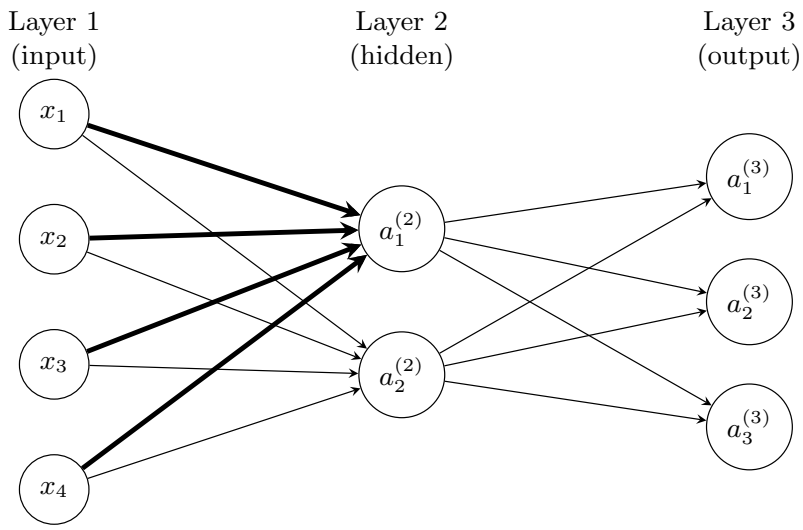
$$W^{(2)} = \begin{bmatrix} w_{1,1}^{(2)} & w_{1,2}^{(2)} & w_{1,3}^{(2)} & w_{1,4}^{(2)} \\ w_{2,1}^{(2)} & w_{2,2}^{(2)} & w_{2,3}^{(2)} & w_{2,4}^{(2)} \end{bmatrix} \qquad W^{(3)} = \begin{bmatrix} w_{1,1}^{(3)} & w_{1,2}^{(3)} \\ w_{2,1}^{(3)} & w_{2,2}^{(3)} \\ w_{3,1}^{(3)} & w_{3,2}^{(3)} \end{bmatrix}$$

Aktiveringsfunksjonen for lag l blir notert med f_l.

I de neste oppgavene vil vi gå igjennom foroverfasen i nevrale nettverk steg for steg.

Oppgave 7.2.1 - Aktiveringene for andre lag

Vi begynner med aktiveringene i andre lag. Vi bruker symbolet z for den *vektede summen*, tilsvarende som med logistisk regresjon og perseptron. For å regne ut aktiveringene begynner vi først med den vektede summen, som er lik aktiveringene fra forrige lag ganget med tilsvarende vekter plusset på med tilsvarende konstantledd. Dersom dette sendes igjennom aktiveringsfunksjonen, får vi aktiveringen for neste lag. Dette kan visualiseres som i Figur 5.



Figur 5: Aktivering av en node i det skjulte laget.

Dette kan regnes ut slik:

$$a_1^{(2)} = f_2\left(z_1^{(2)}\right) = f_2\left(\sum_{k=1}^{4} w_{1,k}^{(2)}\, x_i + b_1^{(2)}\right)$$

$$= f_2\left(w_{1,1}^{(2)}\, x_1 + w_{1,2}^{(2)}\, x_2 + w_{1,3}^{(2)}\, x_3 + w_{1,4}^{(2)}\, x_4 + b_1^{(2)}\right)$$

La oss nå sette noen spesifikke verdier på parameterne våre og prøve å regne litt. La inputen, vektene og konstantleddene for andre lag være gitt ved:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ -1 \\ 3 \end{bmatrix} \qquad W^{(2)} = \begin{bmatrix} 2 & 1 & -2 & 0 \\ 3 & 0 & 5 & -1 \end{bmatrix} \qquad \mathbf{b}^{(2)} = \begin{bmatrix} 2 \\ -3 \end{bmatrix}$$

a) Regn ut $z_1^{(2)}$, den vektede summen for den første noden i det andre laget. **Hint**: Dette er altså gitt ved $\sum_{k=1}^{4} w_{1,k}^{(2)}\, x_i + b_1^{(2)} = w_{1,1}^{(2)}\, x_1 + w_{1,2}^{(2)}\, x_2 + w_{1,3}^{(2)}\, x_3 + w_{1,4}^{(2)}\, x_4 + b_1^{(2)}$. Bruk verdiene for inputen, vektene og konstantleddet over til å fylle inn i denne formelen.

b) Regn ut $z_2^{(2)}$, den vektede summen for den andre noden i det andre laget. **Hint**: Dette blir tilsvarende som i a), men for vektene og konstantleddet for andre node istedenfor første.

Med de vektede summene trenger vi bare å bruke aktiveringsfunksjonen vår for å få aktiveringene. I dette eksemplet bruker vi aktiveringsfunksjonen *ReLU* (rectified linear unit), som er en av de vanligste aktiveringsfunksjonene. Den er definert ved $\mathrm{ReLU}(x) = \max(x, 0)$. Det vil altså si at alt positiv input forblir det samme, mens det negative inputen blir satt til 0.

c) Regn ut $a_1^{(2)}$, aktiveringen for den første noden i det andre laget. Dette blir altså $a_1^{(2)} = \mathrm{ReLU}(z_1^{(2)})$.

d) Regn ut $a_2^{(2)}$, aktiveringen for den andre noden i det andre laget.

Utregning for nevrale nettverk kan gjøres mye mer kompakt ved å bruke matrise- og vektormultiplikasjon. Dette kan gjøre beregningene enklere å forstå dersom man er vant til denne typen notasjon, men det kan også gjøre det vanskeligere om man ikke er det. Vi prøver å bruke både notasjon for én og én node og den vektoriserte formen, slik at man kan vende seg til notasjonen.

Vi kan uttrykke beregningene over med følgende:

$$\mathbf{a}^{(2)} = f_2(\mathbf{z}^{(2)}) = f_2(W^{(2)}\mathbf{x} + \mathbf{b}^{(2)})$$

Vi slipper å bruke notasjon for sum og indekser, som gjør notasjonen betydelig mer kompakt (og også enklere når man vender seg til det). Her er $W^{(2)}\mathbf{x}$ matrisen $W^{(2)}$ ganget med kolonnevektoren $\mathbf{x}$. Matrise- og vektorregningen kan uttrykkes ved følgende:

$$\begin{aligned} W^{(2)}\mathbf{x} + \mathbf{b}^{(2)} &= \begin{bmatrix} w_{1,1}^{(2)} & w_{1,2}^{(2)} & w_{1,3}^{(2)} & w_{1,4}^{(2)} \\ w_{2,1}^{(2)} & w_{2,2}^{(2)} & w_{2,3}^{(2)} & w_{2,4}^{(2)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} b_1^{(2)} \\ b_2^{(2)} \end{bmatrix} \\ &= \begin{bmatrix} w_{1,1}^{(2)}\, x_1 + w_{1,2}^{(2)}\, x_2 + w_{1,3}^{(2)}\, x_3 + w_{1,4}^{(2)}\, x_4 + b_1^{(2)} \\ w_{2,1}^{(2)}\, x_1 + w_{2,2}^{(2)}\, x_2 + w_{2,3}^{(2)}\, x_3 + w_{2,4}^{(2)}\, x_4 + b_1^{(2)} \end{bmatrix} \end{aligned}$$

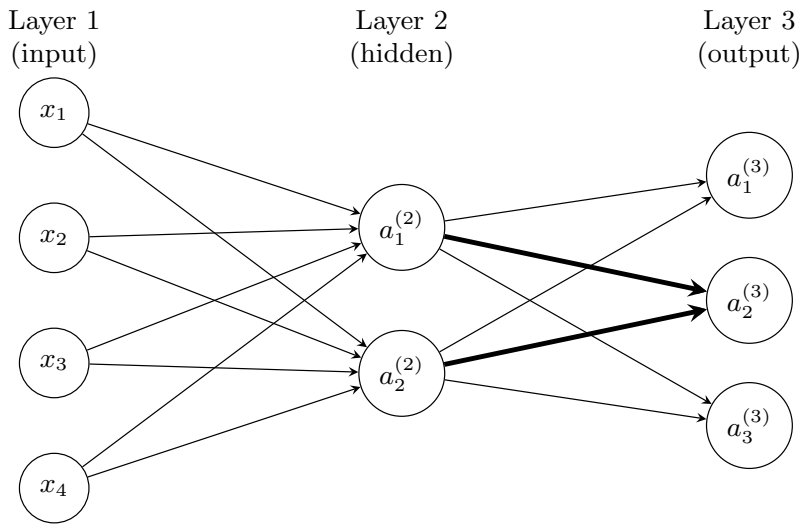
Denne notasjonen, som generelt er mye brukt i matematikk, fysikk, statistikk, robotikk og andre fagområder, er altså veldig passende for utregningene vi trenger med nevrale nettverk.

Siden den vektoriserte formen er mye brukt kan det er lurt å bruke tid på å bli litt vant til den. Implementasjoner av nevrale nettverk i dyplæringsrammeverk er nesten alltid vektorisert, siden dette gir mye mer effektive beregninger.

Dersom dere er interesserte i å lære mer om lineær algebra, er denne videoserien sterkt anbefalt.

Oppgave 7.2.2 - Aktiveringene i tredje lag

Med aktiveringene $\mathbf{a}^{(2)} = \begin{bmatrix} a_1^{(2)} \\ a_2^{(2)} \end{bmatrix}$ kan vi nå regne ut aktiveringene for det tredje laget. Vi kan eksempelvis illustrere dette for den andre noden i tredje lag som i Figur 6.



Figur 6: Aktivering av en node i det siste laget.

Dette gir oss:

$$a_2^{(3)} = f_3\left(z_2^{(3)}\right) = f_3\left(\sum_{j=1}^{2} w_{2,k}^{(3)}\, a_k^{(2)} + b_2^{(3)}\right) = f_3\left(w_{2,1}^{(3)}\, a_1^{(2)} + w_{2,2}^{(3)}\, a_2^{(2)} + b_2^{(3)}\right)$$

Utregningen for det tredje laget blir altså tilsvarende som for det andre laget, der vi bytter ut inputen med aktiveringene fra det andre laget og bruker vekter, konstantledd og aktiveringsfunksjon for lag tre.

La oss bruke følgende vekter og konstantledd for utregningene våre:

$$W^{(3)} = \begin{bmatrix} 2 & 3 \\ -1 & 0 \\ 1 & 2 \end{bmatrix} \qquad \mathbf{b}^{(3)} = \begin{bmatrix} -11 \\ 7 \\ -4 \end{bmatrix}$$

a) Regn ut $z_1^{(3)}$, den vektede summen for den første noden i lag tre. Bruk $\mathbf{a}^{(2)}$ fra oppgave 7.2.1.

b) Regn ut $z_2^{(3)}$.

c) Regn ut $z_3^{(3)}$.

I foroverfasen gjenstår det kun å bruke aktiveringsfunksjonen for det siste laget. I eksemplet vårt har vi tre output-noder, som tilsvarer klassifikasjon med tre klasser. I multiklasse-klassifikasjon brukes funksjonen *softmax*. Denne funksjonen normaliserer inputen litt på samme måte som sigmoid-funksjonen gjør for logistisk regresjon, ved at den gir oss en sannsynlighetsfordeling for de ulike klassene, der hver sannsynlighet er mellom 0 og 1 og sannsynlighetene samlet summerer til 1. Den bevarer også rangeringen av inputene, så dersom $z_i > z_j$ vil $\text{softmax}(z_i) > \text{softmax}(z_j)$. Kombinert med kryssentropi tapsfunksjonen blir også utregningene til de deriverte relativt enkle, tilsvarende som med logistisk regresjon.

For en input $\mathbf{z}$ av lengde n_C er softmax definert i node j ved:

$$\text{softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^{n_c} e^{z_k}}$$

Ved å opphøye inputen z_j i et tall, her den matematiske konstanten e, får vi alltid et positivt output. Ved å dele på summen av alle disse potensene blir outputen normalisert, altså $\sum_{j=1}^{n_c} \text{softmax}(z_j) = 1$.

d) Bruk utregningene fra a), b) og c) til å regne ut $\sum_{j=1}^{3} e^{z_j^{(3)}} = e^{z_1^{(3)}} + e^{z_2^{(3)}} + e^{z_3^{(3)}}$.

e) Regn ut aktiveringen $a_1^{(3)} = \text{softmax}(z_1^{(3)}) = e^{z_1^{(3)}} / \sum_{j=1}^{3} e^{z_j^{(3)}}$.

f) Regn ut $a_2^{(3)}$ og $a_3^{(3)}$ på tilsvarende måte.

Med vektor-notasjon kan dette bli notert tilsvarende som med det forrige laget:

$$\mathbf{a}^{(3)} = f_3(\mathbf{z}^{(3)}) = f_3(W^{(3)}\mathbf{a}^{(2)} + \mathbf{b}^{(3)})$$

der f_3 er sofmax. Matrise og vektor-regningen blir:

$$W^{(3)}\mathbf{a}^{(2)} + \mathbf{b}^{(3)} = \begin{bmatrix} w_{1,1}^{(3)} & w_{1,2}^{(3)} \\ w_{2,1}^{(3)} & w_{2,2}^{(3)} \\ w_{3,1}^{(3)} & w_{3,2}^{(3)} \end{bmatrix} \begin{bmatrix} a_1^{(2)} \\ a_2^{(2)} \end{bmatrix} + \begin{bmatrix} b_1^{(3)} \\ b_2^{(3)} \\ b_3^{(3)} \end{bmatrix} = \begin{bmatrix} w_{1,1}^{(3)} a_1^{(2)} + w_{1,2}^{(3)} a_2^{(2)} + b_1^{(3)} \\ w_{2,1}^{(3)} a_1^{(2)} + w_{2,2}^{(3)} a_2^{(2)} + b_2^{(3)} \\ w_{3,1}^{(3)} a_1^{(2)} + w_{3,2}^{(3)} a_2^{(2)} + b_3^{(3)} \end{bmatrix}$$

Oppgave 7.3 - Kryssentropi tapsfunksjon

For klassifisering brukes ofte tapsfunksjonen *kryssentropi* (cross entropy). I uke 6 så vi på *binær kryssentropi* (binary cross entropy (BCE)), som er formen for kryssentropien med to klasser. For flere enn to klasser er funksjonen essensielt lik, men notasjonen blir litt annerledes. Vi har prediksjoner $\hat{y}$, som ofte er kalt *logits* for nevrale nettverk (som tilsvarer $\mathbf{a}^{(3)}$ fra forrige oppgave) og sanne verdier y. La de sanne verdiene y være *one-hot-enkodet*, det vil si at $\mathbf{y} = (y_1, ..., y_{n_c})$ der alle y_j-ene er 0 bortsett fra indeksen som tilsvarer den sanne klassen. Dersom vi for eksempel har fire klasser og har de sanne verdiene $4, 1$ og 2, vil vi få $(0, 0, 0, 1)$, $(1, 0, 0, 0)$ og $(0, 1, 0, 0)$ one-hot-enkodet. Tapsfunksjonen baserer seg på følgende ledd:

$$-y_j \log(\hat{y}_j)$$

Husk at $\log(1) = 0$ og $\log(x)$ nærmer seg minus uendelig når x nærmer seg 0. Vi ganger først med y_j, som vil være null for alle j bortsett fra j-en som tilsvarer den sanne klassen. For denne indeksen vil tapsfunksjonen være basert på $-\log(\hat{y}_j)$, der $\hat{y}_j$ er et tall mellom 0 og 1 (dette sørger softmax for). Dersom $\hat{y}_j$ er nærme 1, vil $-\log(\hat{y}_j)$ være nærme 0. Dette kan intuitivt gi mening siden det betyr at modellen predikerte en høy verdi for klassen tilsvarende den sanne klassen, så tapet blir lite. Dersom $\hat{y}_j$ er nærme 0, blir $-\log(\hat{y}_j)$ nærme uendelig. Dette gir mening siden det betyr at modellen predikerte en lav verdi for den sanne klassen, så tapet blir høyt.

Gitt ett datapunkt kan vi uttrykke kryssentropi med følgende:

$$\text{cross-entropy}(y, \hat{y}) = -\sum_{j=1}^{n_C} y_j \log(\hat{y}_j)$$

Dette baserer seg altså på leddet $-y_i \log(\hat{y}_i)$ summert over antall klasser n_c.

For n antall datapunkter tar vi gjennomsnittet over alle datapunktene. Da får vi en ytre sum over datapunktene og en indre sum over antall klasser. Vi må ha to indekser, en for datapunktet og en for klassen. Her noterer vi de one-hot-enkodede sanne verdiene med $Y = (\mathbf{y}_1, ..., \mathbf{y}_n)$, der hver $\mathbf{y}_i$ er en one-hot-enkodet verdi $\mathbf{y}_i = (y_{i,1}, ..., y_{i,n_c})$. Tilsvarende har vi logitsene $\hat{Y} = (\hat{\mathbf{y}}_1, ..., \hat{\mathbf{y}}_n)$ der hver $\hat{\mathbf{y}}_i$ er en vektor av lengde n_c (antallet klasser). Vi får:

$$\text{cross-entropy}(Y, \hat{Y}) = -\frac{1}{n} \sum_{i=1}^{n} \sum_{j=1}^{n_C} y_{i,j} \log(\hat{y}_{i,j})$$

Anta vi har prediksjonene (outputen fra softmax) $\hat{y}_1 = (0.3, 0.1, 0.6)$ og $\hat{y}_2 = (0.1, 0.8, 0.1)$ tilsvarende de sanne klassene 3 og 1.

a) Hva blir one-hot-enkodingene av de sanne verdiene?

b) Regn ut kryssentropien tilsvarende det første datapunktet, altså:

$$-\sum_{j=1}^{3} y_{1,j} \log(\hat{y}_{1,j}) = -y_{1,1} \log(\hat{y}_{1,1}) - y_{1,2} \log(\hat{y}_{1,2}) - y_{1,2} \log(\hat{y}_{1,2})$$

c) Regn ut kryssentropien tilsvarende det andre datapunktet.

d) Regn ut kryssentropi-tapet. Dette blir altså gjennomsnittet av verdiene fra de to forrige deloppgavene.

e) Hvorfor kan en modell ikke predikere $\hat{y}_{i,j} = 1$ for alle i og j, og dermed få null i tap? **Hint**: Vi finner $\hat{y}$ ved å bruke softmax.